

LAB 4

Monitoring & production deployment

Course	DDM501 — AI in DevOps, DataOps, MLOps
Weight	15% of the final grade (T4)

1. The scenario

The scenario

Your model went to production in March and passed every test on the way.

In June, a marketing campaign brings in younger applicants with smaller credit limits.

Nothing in your codebase changed. Every test still passes. The service returns well-formed probabilities in 20 milliseconds. And the model is now scoring a population it was never fitted on — so its probabilities mean less than they did, and nobody will know for at least a month.

Why not accuracy?

The obvious answer to "is the model still good?" is to measure its accuracy in production. You cannot.

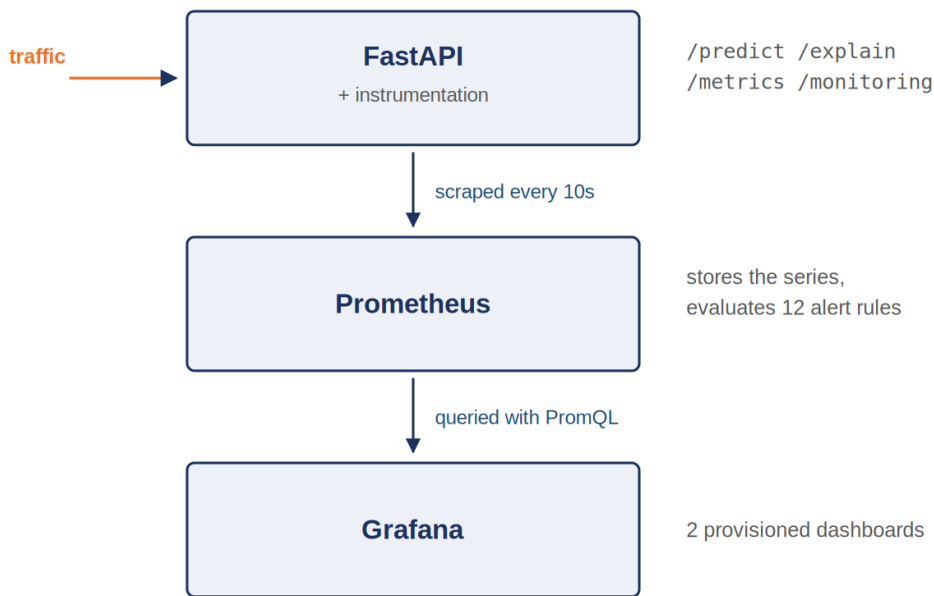
- Whether an applicant defaults is known next month at the earliest. The label arrives long after the decision.
- For an applicant you DECLINED, the label never arrives. You refused them credit, so there is no repayment behaviour to observe. The outcome does not exist.
- By the time an accuracy number is computable, the model has been making decisions on the wrong distribution for weeks.

So production monitoring is a different discipline from model evaluation: watch the signals that move BEFORE accuracy does, and accept that none of them proves the model is wrong. Each is a reason to look.

Signal	The question it answers	Metric in this lab
Input drift	Do the applicants still look like the training population?	ml_feature_drift_psi
Output drift	Has the distribution of scores shifted?	ml_prediction_score
Golden signals	Is the service itself healthy?	http_requests_total

2. What you build

Four containers, one command, and a service that can tell you what it is doing.



No IP address anywhere: Docker's embedded DNS resolves the service names

Figure 1 — the monitoring stack. Four containers, and not one IP address in any config file.

Note what is not in that picture: an IP address. Docker's embedded DNS resolves service names, which is why the scrape target is `api:8000`. Rename the service and the scrape breaks.

Thirteen tasks

#	File	What you write
1	app/monitoring.py	population_stability_index — the PSI formula
2	app/monitoring.py	MonitoringWindow.compute_drift
3	app/monitoring.py	MonitoringWindow.compute_fairness
4	app/monitoring.py	MonitoringWindow.publish
5	app/main.py	_observe — one trap in here
6	app/main.py	GET /metrics
7	app/main.py	GET /monitoring
8	app/main.py	POST /explain
9	app/middleware.py	MetricsMiddleware.dispatch
11	scripts/make_reference.py	build_reference
12	monitoring/prometheus/alerts/ml_alerts.yml	Seven alert rules

#	File	What you write
13	monitoring/grafana/dashboards/model-behaviour.json	The dashboard

What you are given

- app/metrics.py — every metric declared, with the reason for its type written next to it. You should be able to defend each choice.
- monitoring/prometheus/alerts/api_alerts.yml — five service-level rules, the model for the seven you write.
- monitoring/grafana/dashboards/service-health.json — a complete dashboard.
- monitoring/prometheus/tests/alert_tests.yml — promtool unit tests for the rules you have not written yet.
- tests/ They are the specification. Read the test before you write the function; each one names the failure it exists to prevent.

3. Setup

```
cd ddm501-lab4-starter
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

python scripts/make_dataset.py          # 30,000 rows, UCI credit-default schema
python scripts/train_model.py          # ROC AUC ≈ 0.747
```

Why the dataset is generated

The UCI "Default of Credit Card Clients" archive is unreachable from the university network. scripts/make_dataset.py generates data matching that schema exactly — same 23 columns, same encodings, ~23% default rate — so every command in this manual works offline. If your network permits it, scripts/download_data.py fetches the real file and everything downstream is unchanged.

Docker

Once the tasks are done and the reference exists:

```
python scripts/make_reference.py
docker compose up -d --build          # or: make up
```

What	Where
API documentation	http://localhost:8000/docs
Raw metrics	http://localhost:8000/metrics

What	Where
Monitoring as JSON	http://localhost:8000/monitoring
Prometheus	http://localhost:9090
Grafana	http://localhost:3000 — admin / admin

4. Instrumenting the service

Four metric types

Type	Use it for	Getting it wrong
Counter	Totals that only increase: requests, predictions, errors.	A Counter that decreases is read by Prometheus as a process restart. Every rate() over that window is silently corrupted.
Gauge	A current value that moves both ways: drift score, model loaded.	A Gauge where you wanted a Counter loses every increment between scrapes.
Histogram	A distribution: latency, prediction scores.	Buckets spread evenly instead of around the SLO make the one percentile you care about un-estimable.
Info	Static labels: model version, model type.	Putting a version in a metric NAME instead of a label makes every query version-specific.

Naming

- `_total` on counters. Tooling, dashboards and half the Grafana ecosystem assume it.
- Base units, always: `_seconds`, never `_ms`. A metric in milliseconds forces every dashboard and alert to convert, and one of them eventually forgets.
- A namespace prefix — `ml_` here — so model metrics are separable from HTTP ones in a single query.

These conventions are tested. `tests/test_metrics.py` parses `app/metrics.py` with `ast` and asserts every Counter ends in `_total` — checked against the source, because `prometheus_client` normalises the name on the way out.

Buckets are a decision

```
REQUEST_LATENCY = Histogram(
    "http_request_duration_seconds",
    "HTTP request latency",
    ["method", "endpoint"],
```

```
    buckets=[0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1.0, 2.5, 5.0],  
    )
```

There is a boundary at exactly 0.1 because the SLO is 100 ms. `histogram_quantile` only interpolates between boundaries it has; without one at the SLO, "how many requests missed 100 ms" is unanswerable.

TASK 1 — the middleware

Every request passes through the middleware, so it is the one place that can record latency and status for endpoints nobody instrumented. Four things it must get right:

1. Skip `/metrics`. Prometheus scrapes it every ten seconds; counting those dominates the traffic panel on a quiet service.
2. Label with the route TEMPLATE, not the resolved path. `/applications/{id}` labelled by URL mints one series per id, and the cardinality explosion takes Prometheus down — the monitoring fails, not the service.
3. Record in a `finally` block, so a request that raises is still counted. Middleware that records only on success reports a healthy service during an outage.
4. Keep the in-progress gauge balanced: increment before, decrement in the finally. An unbalanced gauge drifts upward forever and eventually alerts on nothing.

5. Drift

PSI

Population Stability Index is the standard drift measure in consumer credit. Bin a feature into k buckets; compare the reference proportion of each against the live one:

PSI	Reading	What to do
< 0.10	No meaningful shift	Nothing.
$0.10 - 0.25$	Moderate shift	Investigate. Is there a known cause?
≥ 0.25	Significant shift	Act. Retraining is likely warranted.

PSI is symmetric, bounded below at zero, and cheap. Its weakness is that it is univariate: income and age can both look normal while their correlation has inverted, and PSI scores zero.

TASK 2 — two decisions

Quantile bins, not equal width

Equal-width bins on a skewed feature put most of the mass in one bucket, and PSI barely moves however far the distribution shifts. Quantile bins start equally populated, so any shift redistributes mass visibly.

The reference is frozen, and belongs to the model

Compute it once, from the training split, and ship it with the model. Recompute it from recent traffic and slow drift becomes permanently invisible: the baseline walks along with the data.

Open the outer edges

The first and last bin edges are $-\text{inf}$ and $+\text{inf}$. A value beyond the training range is exactly the drift most worth seeing; with closed edges `np.histogram` drops it silently.

TASK 3 — the trap

`/predict` receives twenty-three raw columns, but three of the six monitored features are derived inside the pipeline. Record the raw frame and those three arrive as `None`, `dropna()` empties them, and their PSI is pinned at 0.0 forever.

On a dashboard, a monitor measuring nothing looks exactly like a monitor reporting stability. This lab's own solution shipped with that bug, found only because three features reported 0.0000 every run. Hence `sufficient_data` as a field.

6. Alerting

Twelve rules across two files. `api_alerts.yml` is given; you write the seven in `ml_alerts.yml`. Every rule in both obeys the same four disciplines.

Discipline	Why
A for: longer than one scrape	Without it, one unlucky sample pages someone at 3am.
A ratio, not a raw count	An error-rate rule written as a count means something different at 10 rps and at 1000 rps. Written as a ratio of rates it means the same thing at both.
A severity label	"Wake someone" and "look at it tomorrow" are different instructions and must be different labels.
A description saying what to DO	An alert whose recipient has to reverse-engineer the intent at 3am is an alert that gets muted in the second week.

Alert rules are tested

Alert rules execute only during an incident, the worst moment to find a typo in a PromQL expression. `promtool` runs unit tests against the real evaluator:

```
cd monitoring/prometheus/tests
promtool test rules alert_tests.yml
```

```
Unit Testing: alert_tests.yml
SUCCESS
```

Nine cases, two of which assert an alert does NOT fire. A rule suite with no negative cases pages you for nothing, and then somebody silences it.

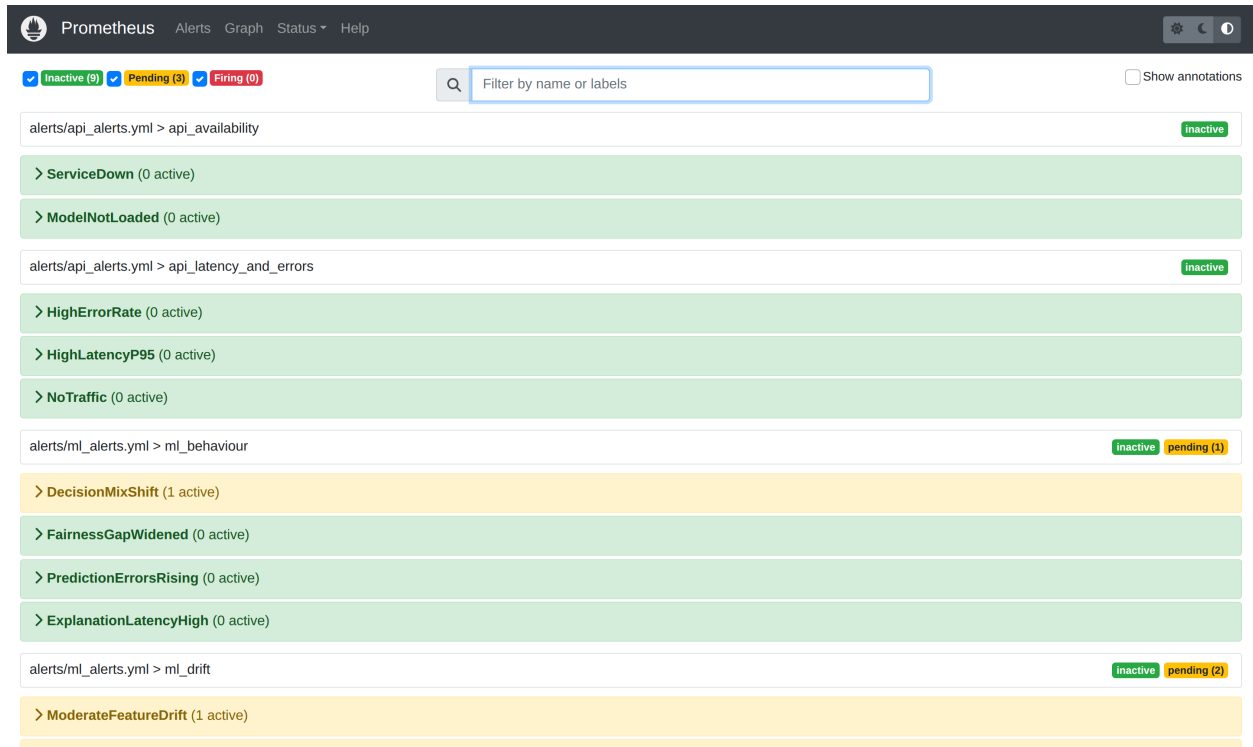


Figure 1 — the rules loaded into Prometheus. Twelve rules, three pending: the `for` window has started but not elapsed.

The ML rules differ

ServiceDown means the service is down. ModerateFeatureDrift does not mean the model is wrong — the arriving applicants no longer match the fitted ones, which may be a campaign, a season, or one upstream system sending a default value. That last is fixed in the pipeline, not by retraining.

Before you retrain on a drift alert

Confirm the drift is real. A constant value arriving in one field produces a textbook drift signature and is a data bug. Check `ml_feature_drift_psi` by feature: one feature at 3.0 while the others sit at 0.01 is almost never a population shift

7. Dashboards

Two, and the split is deliberate. Service Health is the dashboard any HTTP service would have — half of MLOps monitoring is ordinary SRE. Model Behaviour holds the panels that do not exist elsewhere.

Provisioned from files

A dashboard built in the UI exists in one person's container. A dashboard built from a file is in version control and identical everywhere. The datasource has a fixed `uid` so the panels come up populated.

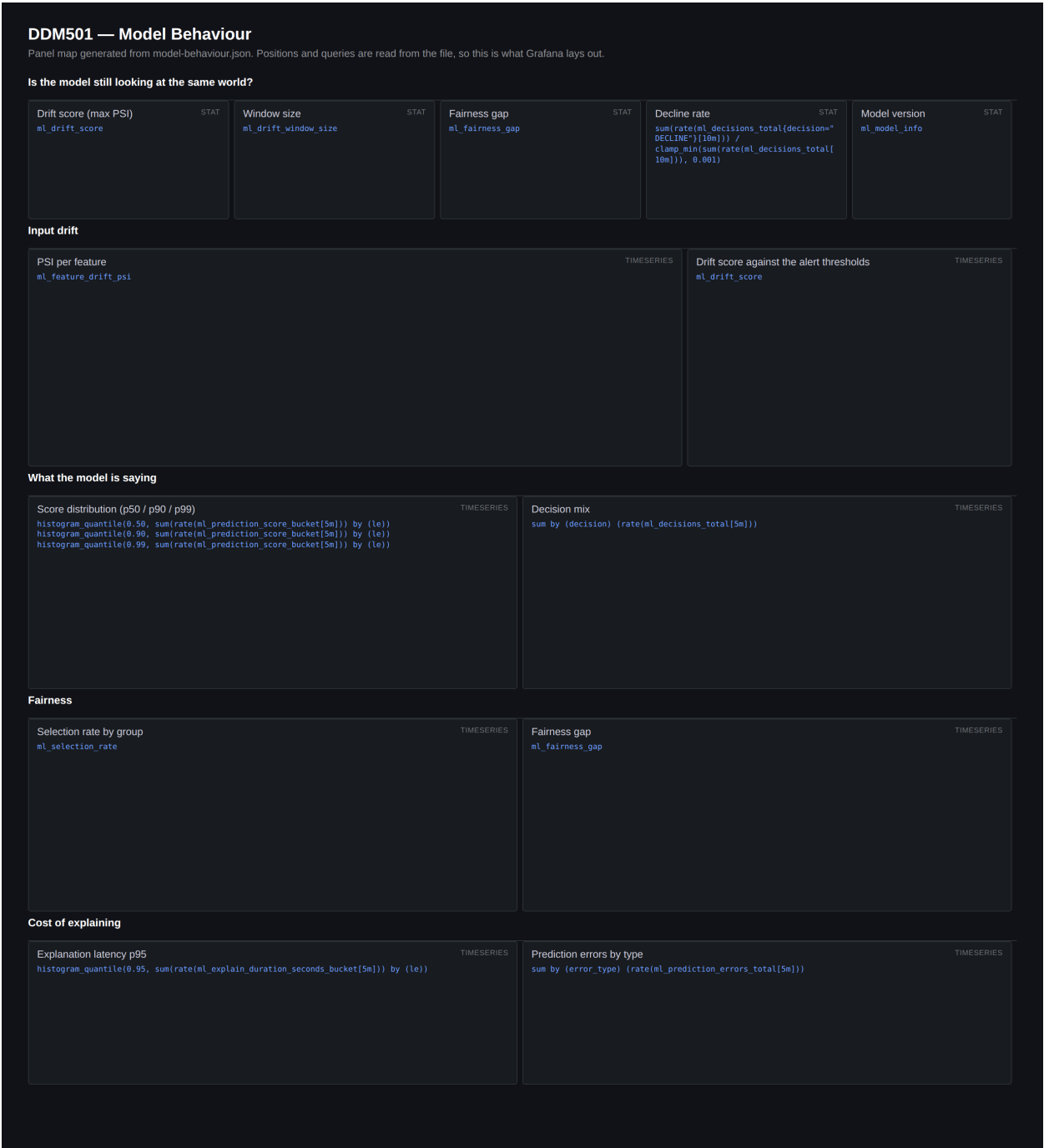


Figure 2 — the Model Behaviour dashboard, rendered from model-behaviour.json. This is TASK 13; one panel is left as a worked example.

What the panels are for

- Drift score and window size, side by side. Below 200 requests the first is noise, and "not measuring" must never look like "nothing wrong".
- PSI per feature, not just the aggregate. The aggregate says something moved; the per-feature panel says what — the difference between an alert and a diagnosis.
- Score distribution as p50/p90/p99 over ml_prediction_score_bucket. This is output drift, observable today, unlike accuracy.
- Decision mix. The business-visible consequence, denominated in manual underwriting work — the panel that makes the ML system legible to people outside the ML team.
- Selection rate by group AND the gap. The rates say what is happening; the gap is what you alert on.

10. Making it happen

A dashboard with no traffic on it teaches nothing. scripts/load_test.py generates three populations against the running service.

Profile	What it does
normal	Applicants drawn from the training distribution. This is what healthy looks like, and you need to have seen it before you can recognise anything else.
drifted	Younger applicants, lower limits, higher utilisation, worse payment history. Nothing is broken. The model is unchanged and still returns well-formed probabilities. Only the INPUTS have moved — the failure no test in Lab 3 can catch, because there is no bug to catch.

The drifted profile takes `--strength` between 0 and 1. Run 0.05, 0.15 and 1.0 and watch PSI climb through the three bands instead of saturating.

```
make load          # normal
make drift-mild    # --strength 0.05
make drift         # full
make unfair

watch -n 2 'curl -s localhost:8000/monitoring | python -m json.tool'
```

Expected results

Measured on the reference solution, 400 requests per profile, the window reset between runs:

Profile	Score mean	REVIEW	DECLINE	Drift score
normal	0.2355	14.2%	8.0%	0.0301 stable
drifted 0.05	0.2437	14.0%	8.5%	0.1885 moderate

Profile	Score mean	REVIEW	DECLINE	Drift score
drifted 0.15	0.2609	16.2%	9.8%	1.1518 significant
drifted full	0.6091	36.8%	57.5%	4.6041 significant
unfair	0.3966	15.8%	32.0%	0.3526 significant

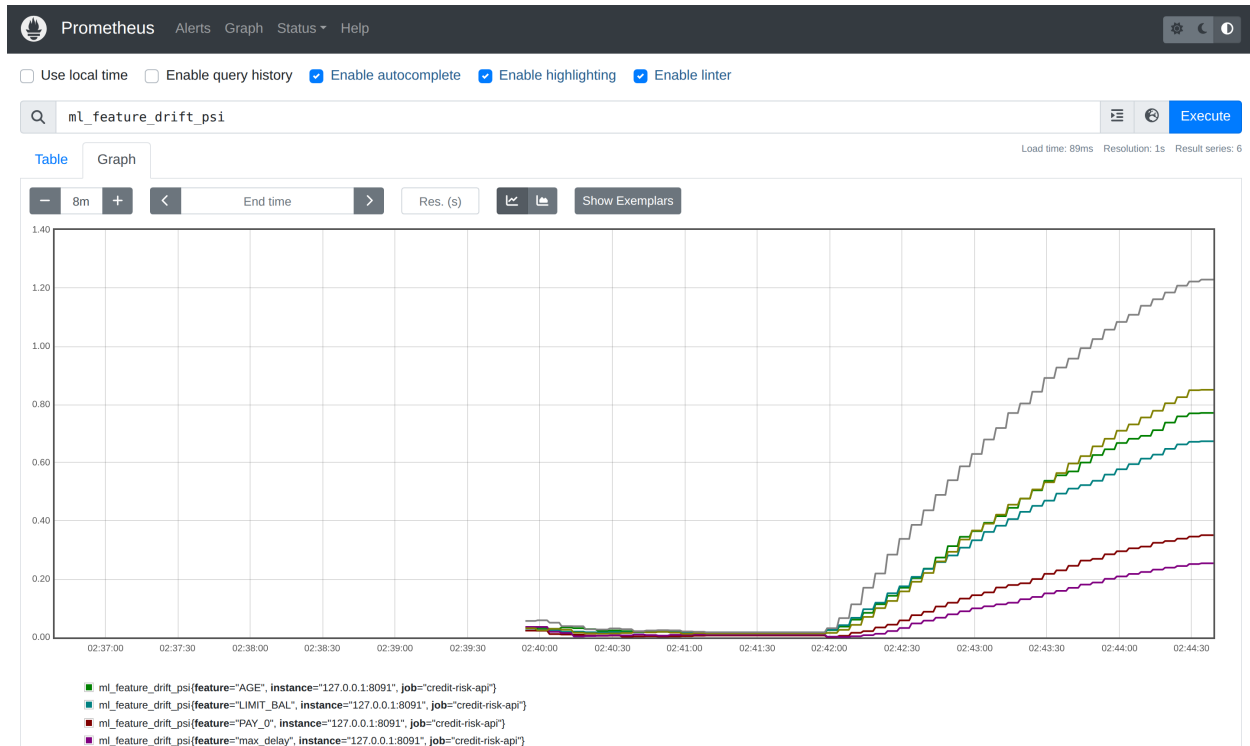


Figure 3 — `ml_feature_drift_psi` during a live run. Flat while normal traffic fills the window, then climbing. `utilisation_ratio` leads, `max_delay` lags: same shift, very different sensitivity.

Read the table

At strength 0.05 the decision mix is indistinguishable from normal and the mean score has moved by eight thousandths. PSI is already 0.19, inside the moderate band.

That gap is the argument for input monitoring. By the time the decline rate visibly moves, you have been scoring the wrong population for weeks.

PSI sensitivity is not uniform. `payment_ratio` reaches the moderate band from a ten percent shift, while `max_delay` sits at 0.0036 through the same run. One global threshold across all features is a blunt instrument.

Drift and fairness are different signals. The unfair run scores 0.35 on drift — so does ordinary drift, and the aggregate does not say WHO absorbed the change. The selection-rate gap does: 0.72 against a 0.019 baseline.

11. Submission

What to submit

- The repository, with all thirteen tasks implemented and CI green.
- The written analysis as a PDF.
- Screenshots of your Model Behaviour dashboard under each of the three profiles.